

4.10 节练习

练习 4.31: 本节的程序使用了前置版本的递增运算符和递减运算符，解释为什么要用前置版本而不用后置版本。要想使用后置版本的递增递减运算符需要做哪些改动？使用后置版本重写本节的程序。

练习 4.32: 解释下面这个循环的含义。

```
constexpr int size = 5;
int ia[size] = {1,2,3,4,5};
for (int *ptr = ia, ix = 0;
     ix != size && ptr != ia+size;
     ++ix, ++ptr) { /* ... */ }
```

练习 4.33: 根据 4.12 节中的表（第 147 页）说明下面这条表达式的含义。

```
someValue ? ++x, ++y : --x, --y
```

4.11 类型转换

159

在 C++ 语言中，某些类型之间有关联。如果两种类型有关联，那么当程序需要其中一种类型的运算对象时，可以用另一种关联类型的对象或值来替代。换句话说，如果两种类型可以**相互转换**（conversion），那么它们就是关联的。



举个例子，考虑下面这条表达式，它的目的是将 `ival` 初始化为 6：

```
int ival = 3.541 + 3; // 编译器可能会警告该运算损失了精度
```

加法的两个运算对象类型不同：3.541 的类型是 `double`，3 的类型是 `int`。C++ 语言不会直接将两个不同类型的值相加，而是先根据类型转换规则设法将运算对象的类型统一后再求值。上述的类型转换是自动执行的，无须程序员的介入，有时甚至不需要程序员了解。因此，它们被称作**隐式转换**（implicit conversion）。

算术类型之间的隐式转换被设计得尽可能避免损失精度。很多时候，如果表达式中既有整数类型的运算对象也有浮点数类型的运算对象，整型会转换成浮点型。在上面的例子中，3 转换成 `double` 类型，然后执行浮点数加法，所得结果的类型是 `double`。

接下来就要完成初始化的任务了。在初始化过程中，因为被初始化的对象的类型无法改变，所以初始值被转换成该对象的类型。仍以这个例子说明，加法运算得到的 `double` 类型的结果转换成 `int` 类型的值，这个值被用来初始化 `ival`。由 `double` 向 `int` 转换时忽略掉了小数部分，上面的表达式中，数值 6 被赋给了 `ival`。

何时发生隐式类型转换

在下面这些情况下，编译器会自动地转换运算对象的类型：

- 在大多数表达式中，比 `int` 类型小的整型值首先提升为较大的整数类型。
- 在条件中，非布尔值转换成布尔类型。
- 初始化过程中，初始值转换成变量的类型；在赋值语句中，右侧运算对象转换成左侧运算对象的类型。
- 如果算术运算或关系运算的运算对象有多种类型，需要转换成同一种类型。
- 如第 6 章将要介绍的，函数调用时也会发生类型转换。



4.11.1 算术转换

算术转换 (arithmetic conversion) 的含义是把一种算术类型转换成另外一种算术类型, 这一点在 2.1.2 节 (第 32 页) 中已有介绍。算术转换的规则定义了一套类型转换的层次, 其中运算符的运算对象将转换成最宽的类型。例如, 如果一个运算对象的类型是 long double, 那么不论另外一个运算对象的类型是什么都会转换成 long double。还有一种更普遍的情况, 当表达式中既有浮点类型也有整数类型时, 整数值将转换成相应的浮点类型。

160

整型提升

整型提升 (integral promotion) 负责把小整数类型转换成较大的整数类型。对于 bool、char、signed char、unsigned char、short 和 unsigned short 等类型来说, 只要它们所有可能的值都能存在 int 里, 它们就会提升成 int 类型; 否则, 提升成 unsigned int 类型。就如我们所熟知的, 布尔值 false 提升成 0、true 提升成 1。

较大的 char 类型 (wchar_t、char16_t、char32_t) 提升成 int、unsigned int、long、unsigned long、long long 和 unsigned long long 中最小的一种类型, 前提是转换后的类型要能容纳原类型所有可能的值。

无符号类型的运算对象

如果某个运算符的运算对象类型不一致, 这些运算对象将转换成同一种类型。但是如果某个运算对象的类型是无符号类型, 那么转换的结果就要依赖于机器中各个整数类型的相对大小了。

像往常一样, 首先执行整型提升。如果结果的类型匹配, 无须进行进一步的转换。如果两个 (提升后的) 运算对象的类型要么都是带符号的、要么都是无符号的, 则小类型的运算对象转换成较大的类型。

如果一个运算对象是无符号类型、另外一个运算对象是带符号类型, 而且其中的无符号类型不小于带符号类型, 那么带符号的运算对象转换成无符号的。例如, 假设两个类型分别是 unsigned int 和 int, 则 int 类型的运算对象转换成 unsigned int 类型。需要注意的是, 如果 int 型的值恰好为负值, 其结果将以 2.1.2 节 (第 32 页) 介绍的方法转换, 并带来该节描述的所有副作用。

剩下的一种情况是带符号类型大于无符号类型, 此时转换的结果依赖于机器。如果无符号类型的所有值都能存在该带符号类型中, 则无符号类型的运算对象转换成带符号类型。如果不能, 那么带符号类型的运算对象转换成无符号类型。例如, 如果两个运算对象的类型分别是 long 和 unsigned int, 并且 int 和 long 的大小相同, 则 long 类型的运算对象转换成 unsigned int 类型; 如果 long 类型占用的空间比 int 更多, 则 unsigned int 类型的运算对象转换成 long 类型。

理解算术转换

要想理解算术转换, 办法之一就是研究大量的例子:

bool	flag;	char	cval;
short	sval;	unsigned short	usval;
int	ival;	unsigned int	uival;
long	lval;	unsigned long	ulval;
float	fval;	double	dval;

```

3.14159L + 'a';      // 'a' 提升成 int, 然后该 int 值转换成 long double
dval + ival;         // ival 转换成 double
dval + fval;         // fval 转换成 double
ival = dval;         // dval 转换成 (切除小数部分后) int
flag = dval;         // 如果 dval 是 0, 则 flag 是 false, 否则 flag 是 true
cval + fval;         // cval 提升成 int, 然后该 int 值转换成 float
sval + cval;         // sval 和 cval 都提升成 int
cval + lval;         // cval 转换成 long
ival + ulval;        // ival 转换成 unsigned long
usval + ival;        // 根据 unsigned short 和 int 所占空间的大小进行提升
uival + lval;        // 根据 unsigned int 和 long 所占空间的大小进行转换

```

161

在第一个加法运算中, 小写字母 'a' 是 char 型的字符常量, 它其实能表示一个数字值 (参见 2.1.1 节, 第 30 页)。到底这个数字值是多少完全依赖于机器上的字符集, 在我们的环境中, 'a' 对应的数字值是 97。当把 'a' 和一个 long double 类型的数相加时, char 类型的值首先提升成 int 类型, 然后 int 类型的值再转换成 long double 类型。最终我们把这个转换后的值与那个字面值相加。最后的两个含有无符号类型值的表达式也比较有趣, 它们的结果依赖于机器。

4.11.1 节练习

练习 4.34: 根据本节给出的变量定义, 说明在下面的表达式中将发生什么样的类型转换:

(a) if (fval) (b) dval = fval + ival; (c) dval + ival * cval;

需要注意每种运算符遵循的是左结合律还是右结合律。

练习 4.35: 假设有如下的定义,

```

char cval;      int ival;      unsigned int ui;
float fval;     double dval;

```

请回答在下面的表达式中发生了隐式类型转换吗? 如果有, 指出来。

(a) cval = 'a' + 3; (b) fval = ui - ival * 1.0;
(c) dval = ui * fval; (d) cval = ival + fval + dval;

4.11.2 其他隐式类型转换



除了算术转换之外还有几种隐式类型转换, 包括如下几种。

数组转换成指针: 在大多数用到数组的表达式中, 数组自动转换成指向数组首元素的指针:

```

int ia[10];      // 含有 10 个整数的数组
int* ip = ia;    // ia 转换成指向数组首元素的指针

```

当数组被用作 decltype 关键字的参数, 或者作为取地址符 (&)、sizeof 及 typeid (第 19.2.2 节, 732 页将介绍) 等运算符的运算对象时, 上述转换不会发生。同样的, 如果用一个引用来初始化数组 (参见 3.5.1 节, 第 102 页), 上述转换也不会发生。我们将在 6.7 节 (第 221 页) 看到, 当在表达式中使用函数类型时会发生类似的指针转换。

指针的转换: C++ 还规定了几种其他的指针转换方式, 包括常量整数值 0 或者字面值 nullptr 能转换成任意指针类型; 指向任意非常量的指针能转换成 void*; 指向任意对象的指针能转换成 const void*。15.2.2 节 (第 530 页) 将要介绍, 在有继承关系的类

162

型间还有另外一种指针转换的方式。

转换成布尔类型：存在一种从算术类型或指针类型向布尔类型自动转换的机制。如果指针或算术类型的值为 0，转换结果是 false；否则转换结果是 true：

```
char *cp = get_string();
if (cp) /* ... */      // 如果指针 cp 不是 0，条件为真
while (*cp) /* ... */ // 如果 *cp 不是空字符，条件为真
```

转换成常量：允许将指向非常量类型的指针转换成指向相应的常量类型的指针，对于引用也是这样。也就是说，如果 T 是一种类型，我们就能将指向 T 的指针或引用分别转换成指向 const T 的指针或引用（参见 2.4.1 节，第 54 页和 2.4.2 节，第 56 页）：

```
int i;
const int &j = i;      // 非常量转换成 const int 的引用
const int *p = &i;     // 非常量的地址转换成 const 的地址
int &r = j, *q = p;    // 错误：不允许 const 转换成非常量
```

相反的转换并不存在，因为它试图删除掉底层 const。

类类型定义的转换：类类型能定义由编译器自动执行的转换，不过编译器每次只能执行一种类型类型的转换。在 7.5.4 节（第 263 页）中我们将看到一个例子，如果同时提出多个转换请求，这些请求将被拒绝。

我们之前的程序已经使用过类类型转换：一处是在需要标准库 string 类型的地方使用 C 风格字符串（参见 3.5.5 节，第 111 页）；另一处是在条件部分读入 istream：

```
string s, t = "a value";    // 字符串字面值转换成 string 类型
while (cin >> s)           // while 的条件部分把 cin 转换成布尔值
```

条件 (cin>>s) 读入 cin 的内容并将 cin 作为其求值结果。条件部分本来需要一个布尔类型的值，但是这里实际检查的是 istream 类型的值。幸好，IO 库定义了从 istream 向布尔值转换的规则，根据这一规则，cin 自动地转换成布尔值。所得的布尔值到底是什么由输入流的状态决定，如果最后一次读入成功，转换得到的布尔值是 true；相反，如果最后一次读入不成功，转换得到的布尔值是 false。

4.11.3 显式转换

有时我们希望显式地将对象强制转换成另外一种类型。例如，如果想在下面的代码中执行浮点数除法：

```
int i, j;
double slope = i/j;
```

就要使用某种方法将 i 和/或 j 显式地转换成 double，这种方法称作**强制类型转换**(cast)。



虽然有时不得不使用强制类型转换，但这种方法本质上是非常危险的。

163 命名的强制类型转换

一个命名的强制类型转换具有如下形式：

```
cast-name<type>(expression);
```

其中，type 是转换的目标类型而 expression 是要转换的值。如果 type 是引用类型，则结果是左值。cast-name 是 static_cast、dynamic_cast、const_cast 和

`reinterpret_cast` 中的一种。`dynamic_cast` 支持运行时类型识别，我们将在 19.2 节（第 730 页）对其做更详细的介绍。`cast-name` 指定了执行的是哪种转换。

`static_cast`

任何具有明确定义的类型转换，只要不包含底层 `const`，都可以使用 `static_cast`。例如，通过将一个运算对象强制转换成 `double` 类型就能使表达式执行浮点数除法：

```
// 进行强制类型转换以便执行浮点数除法
double slope = static_cast<double>(j) / i;
```

当需要把一个较大的算术类型赋值给较小的类型时，`static_cast` 非常有用。此时，强制类型转换告诉程序的读者和编译器：我们知道并且不在乎潜在的精度损失。一般来说，如果编译器发现一个较大的算术类型试图赋值给较小的类型，就会给出警告信息；但是当我们执行了显式的类型转换后，警告信息就会被关闭了。

`static_cast` 对于编译器无法自动执行的类型转换也非常有用。例如，我们可以使用 `static_cast` 找回存在于 `void*` 指针（参见 2.3.2 节，第 50 页）中的值：

```
void* p = &d;    // 正确：任何非常量对象的地址都能存入 void*
// 正确：将 void* 转换回初始的指针类型
double *dp = static_cast<double*>(p);
```

当我们把指针存放在 `void*` 中，并且使用 `static_cast` 将其强制转换回原来的类型时，应该确保指针的值保持不变。也就是说，强制转换的结果将与原始的地址值相等，因此我们必须确保转换后所得的类型就是指针所指的类型。类型一旦不符，将产生未定义的后果。

`const_cast`

`const_cast` 只能改变运算对象的底层 `const`（参见 2.4.3 节，第 57 页）：

```
const char *pc;
char *p = const_cast<char*>(pc); // 正确：但是通过 p 写值是未定义的行为
```

对于将常量对象转换成非常量对象的行为，我们一般称其为“去掉 `const` 性质（`cast away the const`）”。一旦我们去掉了某个对象的 `const` 性质，编译器就不再阻止我们对该对象进行写操作了。如果对象本身不是一个常量，使用强制类型转换获得写权限是合法的行为。然而如果对象是一个常量，再使用 `const_cast` 执行写操作就会产生未定义的后果。

只有 `const_cast` 能改变表达式的常量属性，使用其他形式的命名强制类型转换改变表达式的常量属性都将引发编译器错误。同样的，也不能用 `const_cast` 改变表达式的类型： ◀ 164

```
const char *cp;
// 错误：static_cast 不能转换掉 const 性质
char *q = static_cast<char*>(cp);
static_cast<string>(cp);    // 正确：字符串字面值转换成 string 类型
const_cast<string>(cp);    // 错误：const_cast 只改变常量属性
```

`const_cast` 常常用于有函数重载的上下文中，关于函数重载将在 6.4 节（第 208 页）进行详细介绍。

`reinterpret_cast`

`reinterpret_cast` 通常为运算对象的位模式提供较低层次上的重新解释。举个例

子，假设有如下的转换

```
int *ip;
char *pc = reinterpret_cast<char*>(ip);
```

我们必须牢记 `pc` 所指的真正对象是一个 `int` 而非字符，如果把 `pc` 当成普通的字符指针使用就可能在运行时发生错误。例如：

```
string str(pc);
```

可能导致异常的运行时行为。

使用 `reinterpret_cast` 是非常危险的，用 `pc` 初始化 `str` 的例子很好地证明了这一点。其中的关键问题是类型改变了，但编译器没有给出任何警告或者错误的提示信息。当我们用一个 `int` 的地址初始化 `pc` 时，由于显式地声称这种转换合法，所以编译器不会发出任何警告或错误信息。接下来再使用 `pc` 时就会认定它的值是 `char*` 类型，编译器没法知道它实际存放的是指向 `int` 的指针。最终的结果就是，在上面的例子中虽然用 `pc` 初始化 `str` 没什么实际意义，甚至还可能引发更糟糕的后果，但仅从语法上而言这种操作无可指摘。查找这类问题的原因非常困难，如果将 `ip` 强制转换成 `pc` 的语句和用 `pc` 初始化 `string` 对象的语句分属不同文件就更是如此。



`reinterpret_cast` 本质上依赖于机器。要想安全地使用 `reinterpret_cast` 必须对涉及的类型和编译器实现转换的过程都非常了解。

165

建议：避免强制类型转换

强制类型转换干扰了正常的类型检查（参见 2.2.2 节，第 42 页），因此我们强烈建议程序员避免使用强制类型转换。这个建议对于 `reinterpret_cast` 尤其适用，因为此类类型转换总是充满了风险。在有重载函数的上下文中使用 `const_cast` 无可厚非，关于这一点将在 6.4 节（第 208 页）中详细介绍；但是在其他情况下使用 `const_cast` 也就意味着程序存在某种设计缺陷。其他强制类型转换，比如 `static_cast` 和 `dynamic_cast`，都不应该频繁使用。每次书写了一条强制类型转换语句，都应该反复斟酌能否以其他方式实现相同的目标。就算实在无法避免，也应该尽量限制类型转换值的作用域，并且记录对相关类型的所有假定，这样可以减少错误发生的机会。

旧式的强制类型转换

在早期版本的 C++ 语言中，显式地进行强制类型转换包含两种形式：

```
type (expr);           // 函数形式的强制类型转换
(type) expr;           // C 语言风格的强制类型转换
```

根据所涉及的类型不同，旧式的强制类型转换分别具有与 `const_cast`、`static_cast` 或 `reinterpret_cast` 相似的行为。当我们在某处执行旧式的强制类型转换时，如果换成 `const_cast` 和 `static_cast` 也合法，则其行为与对应的命名转换一致。如果替换后不合法，则旧式强制类型转换执行与 `reinterpret_cast` 类似的功能：

```
char *pc = (char*) ip; // ip 是指向整数的指针
```

的效果与使用 `reinterpret_cast` 一样。



与命名的强制类型转换相比，旧式的强制类型转换从表现形式上来说不那么清晰明了，容易被看漏，所以一旦转换过程出现问题，追踪起来也更加困难。

4.11.3 节练习

练习 4.36: 假设 i 是 int 类型，d 是 double 类型，书写表达式 i*=d 使其执行整数类型的乘法而非浮点类型的乘法。

练习 4.37: 用命名的强制类型转换改写下列旧式的转换语句。

```
int i; double d; const string *ps; char *pc; void *pv;
(a) pv = (void*)ps; (b) i = int(*pc);
(c) pv = &d; (d) pc = (char*) pv;
```

练习 4.38: 说明下面这条表达式的含义。

```
double slope = static_cast<double>(j/i);
```

4.12 运算符优先级表

表 4.4: 运算符优先级

结合律和运算符		功能	用法	参考页码
左	::	全局作用域	::name	256
左	::	类作用域	class::name	79
左	::	命名空间作用域	namespace::name	74
左	.	成员选择	object.member	20
左	->	成员选择	pointer->member	98
左	[]	下标	expr[expr]	104
左	()	函数调用	name(expr_list)	20
左	()	类型构造	type(expr_list)	145
右	++	后置递增运算	lvalue++	131
右	--	后置递减运算	lvalue--	131
右	typeid	类型 ID	typeid(type)	731
右	typeid	运行时类型 ID	typeid(expr)	731
右	explicit cast	类型转换	cast_name<type>(expr)	144
右	++	前置递增运算	++lvalue	131
右	--	前置递减运算	--lvalue	131
右	~	位求反	~expr	136
右	!	逻辑非	!expr	126
右	-	一元负号	-expr	124
右	+	一元正号	+expr	124
右	*	解引用	*expr	48
右	&	取地址	&lvalue	47
右	()	类型转换	(type) expr	145
右	sizeof	对象的大小	sizeof expr	139